

Git Repo History Understanding System 实现计划

0. 项目目标

构建一个用于代码库理解的 Git 历史分析系统。系统将一个 Git repository 编译为结构化历史信息，并在此基础上提供：

- 可复现的 Git 历史抽象
- agent-readable 的验证报告
- 非图形化理解报告
- 可交互的可视化界面
- 与图形对象绑定的持续追问对话框 Context Chat
- 面向代码理解的查询接口

系统的目标不是辅助自动写代码，而是帮助用户和 agent 理解代码库的演化历史。

核心问题包括：

这个仓库如何演化？
主要开发主题是什么？
哪些 stem 像 feature、bugfix、release 或 stale branch？
某次 merge 实际合入了什么？
哪些文件或目录长期高频变化？
某个 release 前后发生了什么？
某个 cluster 的主题判断有什么证据？

1. 总体设计原则

1.1 结构化 IR 是事实源

系统不能依赖 LLM 直接猜测 Git 历史。所有关键事实必须来自确定性 Git 数据处理流程。

```
Git commands / GitHub API
↓
Repository History IR
↓
Validator / Visualization / Context Chat
```

LLM 可以负责解释、总结、命名和辅助探索，但不能替代底层 Git 拓扑分析。

1.2 可视化是 IR 的投影，不是唯一依据

可视化用于帮助用户总体把握信息，但图形本身不是事实源。

IR 负责事实
Visualization 负责定位和总体感知
Context Chat 负责解释和持续追问
Validator Agent 负责检查一致性

用户点击图中的对象时，系统解释的依据不是截图，而是该对象绑定的结构化数据和证据链。

1.3 Evidence-first

每个高层抽象都必须能回到原始证据：

Stem → commits / refs / merge episodes
CSM → base commit / source commits / aggregated context
Cluster → commits / authors / files / keywords / CLOC
Hotspot → files / directories / related commits
Release slice → tags / included commits / merge episodes

系统输出任何解释时，都应包含 supporting commits、files、time range、authors 或 Git command evidence。

1.4 Context Chat 是交互中枢

系统不应只提供静态解释面板，而应提供一个可持续追问的 Context Chat。

用户点击图形对象
↓
UI 捕获 selectedObject
↓
系统读取对应 IR 和证据
↓
LLM 解释该对象
↓
用户继续追问
↓
LLM 可以回答、比较、过滤、高亮或打开证据

用户不需要一次性读懂复杂图，而是可以通过自然语言逐步进入图形和历史结构。

1.5 Agent 检查与用户探索分离

系统中有两类 agent 能力：

1. **Validator Agent**：检查 IR 抽象是否自治、可复现、有证据。
2. **Context Chat Agent**：面向用户解释当前选中对象，支持持续追问和 UI 操作。

两者职责不同：

Validator Agent 负责可靠性
Context Chat Agent 负责理解体验

2. 总体架构

```
Git Repository
  ↓
History Extractor
  ↓
Repository History IR
  ↓
DAG / Stem / Merge / CSM / Cluster Compiler
  ↓
Validator Agent
  ↓
Reports + Query Index
  ↓
Visualization UI
  ↓
Context Chat
```

系统分为七层：

1. **数据抽取层**：提取 commits、parents、authors、messages、dates、files、LOC、tags、branches。
2. **中间表示层**：构建 Repository History IR。
3. **历史编译层**：生成 DAG、stem、segment、merge episode、CSM、cluster、hotspot 等对象。
4. **验证层**：用确定性规则和 agent 检查抽象是否可靠。
5. **报告层**：生成 Markdown/JSON/SQLite 等可读、可查、可复现产物。
6. **可视化层**：将 IR 投影成可交互图形界面。
7. **Context Chat 层**：围绕当前选中对象提供持续追问、解释、比较和 UI 操作。

3. 核心对象模型

3.1 Commit

表示一个 Git commit。

```
{
  "sha": "...",
  "parents": [...],
  "author_name": "...",
```

```
"author_email": "...",
"author_date": "...",
"committer_date": "...",
"subject": "...",
"body": "...",
"refs": ["main", "feature/auth"],
"tags": ["v1.2.0"],
"files": [
  {
    "path": "src/auth/login.ts",
    "added": 20,
    "deleted": 5,
    "cloc": 25
  }
]
```

3.2 DAG

表示 Git commit graph。

```
node = commit
edge = parent-child relationship
```

DAG 是后续 stem、merge episode、CSM、cluster 的基础。

3.3 Stem

Stem 是从 DAG 中抽象出的一条线性开发路径。

它不是 Git branch 的一一映射，而是代码历史理解层面的开发路径抽象。

```
branch 是 Git 概念
stem 是分析概念
```

一个 stem 可能来自：

- main branch
- feature branch
- bugfix branch
- release branch
- pull request
- implicit branch
- stale branch
- long-running maintenance branch

3.4 Stem Segment

一个 stem 可以被拆分为多个 segment。

这用于处理如下情况：

一个 stem 的前半段已经 merge 到 main，
但该 stem 后续仍然继续发展。

因此，merge 不一定意味着 stem 结束。更准确的建模是：

Stem = 开发路径
Stem Segment = 两次关键事件之间的一段提交
Merge Episode = 某个 segment 合入 main 的事件
Unmerged Tail = merge 后继续发展但尚未合入的部分

示例：

```
{
  "stem_id": "stem_feature_auth",
  "source_ref": "feature/auth",
  "type": "feature_candidate",
  "commits": ["D", "E", "F", "G"],
  "lifetime": {
    "start": "2024-03-02",
    "end": "2024-03-18"
  },
  "segments": [
    {
      "segment_id": "seg_1",
      "commits": ["D", "E"],
      "status": "merged",
      "merge_episode": "merge_M1"
    },
    {
      "segment_id": "seg_2",
      "commits": ["F", "G"],
      "status": "unmerged_tail"
    }
  ]
}
```

3.5 Merge Episode

Merge Episode 表示一次合并事件。

```
{
  "merge_episode_id": "merge_M1",
  "base_commit": "M1",
  "first_parent": "C",
  "other_parent": "E",
  "source_commits": ["D", "E"],
  "target_stem": "main",
  "source_stem": "stem_feature_auth",
  "merged_at": "2024-03-08"
}
```

Merge Episode 用于回答：

这次 merge 实际合入了哪些 commits?
这些 commits 来自哪个 stem?
它们是否进入了 main?
这个 stem 后续是否继续发展?

3.6 CSM Capsule

CSM Capsule 表示一次 Context-Preserving Squash Merge 的分析结果。

CSM 不是 Git 中真实创建的新 commit，而是分析层对某个 merge commit 的增强表示。

CSM-base = merge commit
CSM-source = 被合入的 source commits
CSM Capsule = base + source context + aggregated evidence

示例：

```
{
  "csm_id": "csm_M1",
  "base_commit": "M1",
  "source_commits": ["D", "E"],
  "aggregated_authors": ["Alice", "Bob"],
  "aggregated_files": [
    "src/auth/login.ts",
    "src/auth/token.ts"
  ],
  "keywords": ["login", "token", "session"],
  "summary": "Likely authentication login/token work",
  "evidence": {
    "commands": [
      "git rev-list E --not C"
    ]
  },
}
```

```
"representative_commits": [  
  "D add login API",  
  "E implement token refresh"  
]  
}  
}
```

3.7 Commit Cluster

Commit Cluster 表示一组相似 commits，通常对应一个开发主题候选。

```
{  
  "cluster_id": "cluster_auth_01",  
  "stem_id": "stem_feature_auth",  
  "commits": ["D", "E", "F"],  
  "time_range": {  
    "start": "2024-03-02",  
    "end": "2024-03-06"  
  },  
  "top_authors": ["Alice", "Bob"],  
  "top_files": [  
    "src/auth/login.ts",  
    "src/auth/token.ts"  
  ],  
  "top_keywords": ["login", "token", "session"],  
  "cloc": 430,  
  "label": "authentication login/token work",  
  "confidence": "medium"  
}
```

LLM 可以为 cluster 命名，但 cluster 成员必须由确定性或可复现算法产生。

3.8 Hotspot

Hotspot 表示高频或大规模变化的文件/目录。

```
{  
  "hotspot_id": "hotspot_src_auth",  
  "path": "src/auth",  
  "type": "directory",  
  "cloc": 2840,  
  "commit_count": 73,  
  "related_clusters": ["cluster_auth_01", "cluster_auth_02"],  
  "top_authors": ["Alice", "Bob"]  
}
```

3.9 Release Slice

Release Slice 表示围绕某个 release/tag 的历史切片。

```
{
  "release_id": "v1.3",
  "tag_commit": "...",
  "time_range": ["2024-03-01", "2024-03-31"],
  "included_commits": [...],
  "related_stems": [...],
  "related_clusters": [...],
  "related_merge_episodes": [...]}
}
```

4. 数据抽取层：History Extractor

4.1 输入

```
repo_path
main_branch
time_range 可选
include_all_refs 可选
include_numstat 可选
include_tags 可选
```

4.2 输出

```
.repo-history/
raw/
  commits.jsonl
  refs.json
  tags.json
  branches.json
  file_changes.jsonl
```

4.3 实现方式

优先使用本地 Git 命令：

```
git log --all --parents --date=iso-strict --numstat
git for-each-ref
```



```
git tag --points-at  
git branch --contains
```

后续可接入 GitHub/GitLab API，用于补充：

- PR title
- PR body
- PR state
- review comments
- issue links
- release notes
- CI status

5. Repository History IR

5.1 目标

把原始 Git 数据转换为统一中间表示，供后续编译、验证、可视化和 agent 查询使用。

5.2 文件结构

MVP 阶段使用 JSON/JSONL：

```
.repo-history/  
  ir/  
    commits.jsonl  
    dag.json  
    stems.json  
    stem_segments.json  
    merge_episodes.json  
    csm_capsules.json  
    clusters.json  
    hotspots.json  
    ownership.json  
    releases.json
```

后续可加入 SQLite 或 DuckDB：

```
.repo-history/  
  repo_history.duckdb
```

6. Stem Compiler

6.1 目标

从原始 DAG 中抽象出 main stem 和 non-main stems。

6.2 基础规则

1. main stem 来自 main/master 的 first-parent 链。
 2. non-main stems 可来自 branch heads、PR heads、release branches 或隐式分支候选。
 3. 每个 stem 是一条线性 commit path。
 4. 一个 stem 可以包含多个 segment。
 5. stem 被 merge 到 main 后，不一定结束。
 6. merge 后继续发展的 commits 应作为新的 segment 或 unmerged tail。
 7. stem type 只作为 candidate，不做绝对判断。
-

6.3 验证规则

- stem 是否是一条连续 path。
 - main stem 是否覆盖 main branch first-parent 链。
 - commit 时间是否基本符合拓扑顺序。
 - 同一 commit 是否被错误归入多个不兼容 stem。
 - stem lifetime 是否与 first/last commit 对应。
 - stem 被 merge 后继续发展时，是否被正确拆分为 merged segment 和 unmerged tail。
-

7. Merge Episode 与 CSM Compiler

7.1 目标

识别每次 merge，把被 merge 的上下文压缩成可理解、可验证的 CSM Capsule。

7.2 Merge Episode 识别

对于每个 merge commit：

```
base_commit = merge commit
first_parent = parents[0]
other_parent = parents[1...]
source_commits = commits reachable from other_parent but not first_parent
```

可复现命令示例：

```
git rev-list <other-parent> --not <first-parent>
```

7.3 CSM 聚合内容

CSM Capsule 应聚合：

- source commits
- authors
- commit messages
- commit types
- changed files
- changed directories
- CLOC
- keywords
- related PR information
- related release information

7.4 验证规则

- CSM-base 是否是合法 merge commit。
- source commits 是否能通过 Git 命令复现。
- source commits 是否重复归属到多个 CSM。
- CSM 聚合的 author、file、keyword 是否都能追溯到 source commits。
- CSM summary 是否有 commit message 和 file path 支撑。
- 如果 source stem 后续继续发展，CSM 是否只吸收已 merge segment，而不是整个 stem。

8. Commit Clustering

8.1 目标

把零散 commits 聚合成更高层的开发主题块。

8.2 聚类特征

基础特征包括：

- author
- commit date
- commit type
- changed files
- changed directories
- commit message keywords
- CLOC
- stem membership

- merge episode membership
-

8.3 相似度模型

MVP 可使用可解释的加权相似度：

```
similarity =  
    w_author * author_similarity  
+ w_file * file_path_similarity  
+ w_message * message_similarity  
+ w_time * time_proximity  
+ w_type * commit_type_similarity
```

其中：

```
author / file / type 可用 Jaccard similarity  
message 可用 TF-IDF + cosine similarity  
time 可用时间距离衰减函数
```

8.4 Agent 的角色

确定性脚本负责：

- 哪些 commits 属于 cluster
- cluster 的时间范围
- cluster 的文件、作者、CLOC、关键词统计
- cluster 的证据链

LLM 负责：

- 给 cluster 命名
 - 生成解释性 summary
 - 标记可能的 outlier commit
 - 判断 cluster 是否语义过宽或过窄
-

9. Validator Agent

9.1 目标

让 agent 检查历史抽象是否可靠，而不是只让用户通过图形判断。

9.2 两层验证

第一层是 deterministic validator，负责硬事实检查：

```
DAG 是否有效
parent-child 是否正确
merge source set 是否可复现
CSM 是否遗漏或重复 source commits
cluster 是否有过大时间跨度
stem 是否错误重叠
release tag 是否位于预期路径
CLOC 是否可复现
```

第二层是 LLM reviewer agent，负责语义合理性检查：

```
stem 的类型判断是否合理
cluster label 是否有证据
CSM summary 是否遗漏重要上下文
hotspot 是否真的值得高亮
是否存在“看似 feature，实际只是同步分支”的情况
```

9.3 输出

```
.repo-history/
validation/
  validation_report.md
  validation_findings.json
  warnings.jsonl
```

9.4 示例报告

```
Stem S12: feature-auth
Status: WARN

PASS first-parent path is continuous
PASS merge episode M1 is reproducible
WARN stem continues after M1; split into merged segment and unmerged tail
PASS cluster C3 has strong file-path cohesion
WARN cluster C4 spans 42 days; consider splitting

Evidence:
- 21 commits
- 16 commits touch src/auth/*
- top keywords: login, token, session
```

- merged segment: D,E via M1
- unmerged tail: F,G,H

10. 非图形报告

10.1 目标

先实现可读报告，验证系统价值，不立即依赖复杂 UI。

10.2 报告文件

```
.repo-history/  
reports/  
  repo_overview.md  
  stems.md  
  stem_segments.md  
  merge_episodes.md  
  csm_capsules.md  
  clusters.md  
  hotspots.md  
  releases.md  
  validation_report.md
```

10.3 repo_overview.md 内容

- 仓库时间范围
 - commit 总数
 - 作者数量
 - 文件数量
 - main stem 概览
 - non-main stem 数量
 - 最大 cluster
 - 最大 hotspot 文件/目录
 - release 时间线
 - 最近活跃 stems
 - stale stems
 - 高风险历史区域
 - 低置信度解释
 - validator warnings
-

11. 可视化设计

11.1 目标

可视化用于总体把握信息，但不应要求用户一次性读懂复杂图。

设计原则：

不要让用户读完整复杂图
而是让用户通过问题、卡片、点击和 Context Chat 逐步进入图

11.2 可视化模式

Overview Mode

第一屏不展示完整复杂 stem graph，而是展示仓库历史概览卡片：

历史范围
主要 release
主要开发阶段
主要 stems
疑似 feature stems
疑似 bugfix stems
stale stems
热点目录
最近一次大规模变化
validator warnings

Explore Mode

用户点击某个 overview card、stem、cluster 或 hotspot 后，图上只突出相关对象。

高亮 selected object
弱化其他对象
右侧 Context Chat 自动解释
底部显示 evidence table

Evidence Mode

每个图形对象都可以展开为证据：

Stem → commits / merge episodes / related files
Cluster → authors / messages / changed files / keywords
CSM-base → CSM-source commits
Hotspot → changed files / CLOC / related commits
Release → included commits / related clusters

Compare Mode

用户可以比较两个对象：

release vs release
stem vs stem
cluster vs cluster
hotspot vs hotspot

比较维度包括：

- authors
- commit types
- files
- directories
- keywords
- CLOC
- time range
- merge episodes

11.3 Zoom Levels

可视化应支持渐进式展开：

Level 0: 只看阶段 / release / 大 cluster
Level 1: 显示主要 stems
Level 2: 显示 cluster 和 CSM-base
Level 3: 显示 merge episode / CSM-source
Level 4: 显示 commit list 和 file changes

默认停留在 Level 0 或 Level 1。

11.4 视觉编码

横向：时间
纵向：stem lane
矩形：cluster 或 stem segment

矩形宽度：时间跨度或 commit 数
矩形高度：commit 数或 CLOC
边框：是否包含 CSM-base
颜色：stem 类型或 cluster 类型
虚线：merge / CSM 关系
细 strip：区分同一 lane 中不同 stem

11.5 Stem Lane 布局

垂直方向的 level 不等于 Git branch，也不严格等于 stem。

更准确地说：

水平 level = 可复用的视觉 lane
stem = 在某段时间内占用 lane 的对象

布局规则：

1. main stem 固定在最上方。
2. non-main stems 默认按最近活动时间排序。
3. stem lifetime 为 `[firstCommitTime, lastCommitTime]`。
4. 时间不重叠的 stems 可以复用同一 lane。
5. 如果 stem 被 merge 后继续发展，其后续 segment 仍应保留在该 stem 语义下，但可作为新 segment 显示。
6. 如果视觉上复用 lane，必须用 strip、label 或 hover tooltip 区分不同 stems。

12. Context Chat

12.1 目标

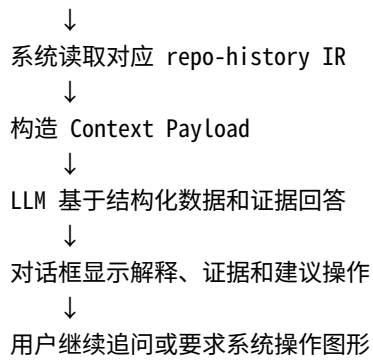
Context Chat 是与可视化对象绑定的持续追问对话框。

它不是普通聊天框，而是代码历史理解系统的交互中枢。

图形负责定位对象
结构化数据负责提供事实
对话框负责解释、追问、比较和操作图形

12.2 核心流程

用户点击图形对象
↓
UI 捕获 `selectedObject = { type, id }`



12.3 Context Payload

每次用户提问时，前端发送：

```
{
  "question": "它后来合入 main 了吗？ ",
  "selected_object": {
    "type": "cluster",
    "id": "cluster_auth_01"
  },
  "view_state": {
    "time_range": ["2024-03-01", "2024-04-01"],
    "filters": ["path:src/auth"],
    "zoom_level": "cluster"
  },
  "pinned_objects": [
    {
      "type": "release",
      "id": "v1.3"
    }
  ],
  "conversation_state": {
    "last_selected_object": "cluster_auth_01",
    "recent_topics": ["auth", "merge", "release"]
  }
}
```

后端根据 selected object 补充证据：

```
{
  "selected_object_detail": {
    "type": "cluster",
    "id": "cluster_auth_01",
    "time_range": ["2024-03-02", "2024-03-12"],
    "commit_count": 18,
    "cloc": 430,
  }
}
```

```
"top_authors": ["Alice", "Bob"],
"top_files": [
  "src/auth/login.ts",
  "src/auth/token.ts"
],
"top_keywords": ["login", "token", "session"],
"related_stem": "stem_feature_auth",
"related_merge_episodes": ["merge_M1"],
"related_csm_capsules": ["csm_M1"]
},
"evidence": {
  "representative_commits": [
    "D add login API",
    "E implement token refresh",
    "F fix session expiration"
  ],
  "related_files": [
    "src/auth/login.ts",
    "src/auth/token.ts"
  ]
}
}
```

12.4 支持的问题类型

解释类

这条 stem 是什么？
为什么这个 cluster 被认为是 auth 相关？
这个 CSM-base 实际合入了什么？

追踪类

它后来合入 main 了吗？
这个 stem 后面还有 unmerged tail 吗？
这个文件最早在哪个 cluster 中出现？

比较类

把这个 cluster 和上一个 release 比较一下。
这个 stem 和旁边那个 stem 有什么关系？
v1.2 和 v1.3 之间主要差异是什么？

过滤类

只看和这个 cluster 相关的 commits。
隐藏无关 stems。
只显示 src/auth 相关历史。

证据类

展开 supporting commits。
显示这个判断的依据。
哪些文件支撑这个主题判断？

12.5 Agent 可返回的 UI Actions

Context Chat 不只返回自然语言，也可以返回结构化 UI 操作。

示例：

```
{
  "answer": "这个 cluster 通过 merge_M1 合入了 main。相关 source commits 包括 D、E、F。",
  "evidence": [
    "D",
    "E",
    "F",
    "merge_M1"
  ],
  "actions": [
    {
      "type": "highlight",
      "object_ids": ["cluster_auth_01", "merge_M1", "csm_M1"]
    },
    {
      "type": "open_evidence",
      "object_id": "merge_M1"
    }
  ]
}
```

MVP 支持的 actions：

```
highlight(object_ids)
focus(object_id)
filter_by_object(object_id)
open_evidence(object_id)
open_comparison(object_id_a, object_id_b)
```

```
pin_object(object_id)
clear_filters()
```

12.6 UI 布局

将右侧区域设计为 Context Chat:

Repo Overview Cards	
Simplified Stem Timeline	Context Chat
main stem	当前对象
selected stems/clusters	持续追问
highlighted evidence	推荐问题
Evidence Table / Commit List / File Summary	

Context Chat 顶部应显示当前上下文:

```
Current Object: cluster_auth_01
Pinned: stem_feature_auth, release_v1.3
Current Filter: path:src/auth
```

12.7 Suggested Questions

当用户点击某个对象时，系统自动生成 suggested questions。

点击 stem:

```
这条 stem 主要做了什么？
它是否已经合入 main？
它有没有 unmerged tail？
它和哪些 release 相关？
```

点击 cluster:

```
这个 cluster 的主题是什么？
这个主题判断有什么证据？
它主要修改了哪些文件？
它属于哪个 stem？
```

点击 CSM-base:

这次 merge 实际合入了什么?
source commits 有哪些?
哪些作者参与了这次 merge?
这次 merge 是否关联 PR?

点击 hotspot:

为什么这个文件是 hotspot?
哪些 cluster 修改了它?
它在哪些 release 前后变化最大?
它的主要作者是谁?

12.8 可靠性要求

Context Chat 必须遵循 evidence-first 原则:

1. 不允许只基于视觉位置做事实判断。
2. 必须优先使用 repo-history IR 中的结构化数据。
3. 涉及 stem、cluster、CSM、merge episode 的解释必须附 supporting commits/files。
4. 如果证据不足, 必须标记 low confidence。
5. 如果问题涉及当前截图布局, 可将截图作为辅助输入, 但事实判断必须来自结构化数据。

推荐回答格式:

结论:
这个 cluster 很可能是一组 auth/login 相关开发活动。

依据:

- 18 个 commits 中, 多数修改 src/auth/*
- 高频关键词包括 login、token、session
- 主要作者是 Alice 和 Bob
- 它通过 merge_M1 合入 main

不确定性:

- commit message 较短, 主题标签置信度为 medium

13. Agent Query Interface

13.1 目标

让 agent 可以基于 repo history IR 回答代码理解问题。

13.2 示例问题

这个仓库最近主要有哪些开发主题？
哪些 stem 看起来像长期维护分支？
这个 release 合入了哪些主要功能？
哪个目录历史上最活跃？
某个文件为什么频繁变化？
某个 cluster 的证据是什么？
这个 merge commit 实际合入了哪些上下文？

13.3 查询方式

MVP:

读取 JSON + Markdown 报告

增强版:

SQLite / DuckDB 查询

最终版:

结构化查询 + embedding retrieval + agent reasoning

14. Codex Skill 形态

系统可以作为 Codex skill 或类似 agent skill 分发。

建议目录结构:

```
.agents/skills/repo-history-understanding/  
  SKILL.md  
  scripts/  
    extract_git.py  
    build_dag.py  
    build_stems.py  
    compute_merge_episodes.py  
    compute_csm.py  
    cluster_commits.py  
    validate_history.py  
    generate_reports.py  
  references/
```

```
schema.md
interpretation_guide.md
validation_rules.md
assets/
report_template.md
```

SKILL.md 应定义：

```
何时使用该 skill
如何运行 history extractor
如何生成 IR
如何运行 validator
如何读取报告
如何回答代码历史理解问题
```

15. 技术选型

15.1 后端 / 编译器

MVP：

```
Python
Git CLI
JSON / JSONL
```

增强版：

```
DuckDB or SQLite
NetworkX 可选
scikit-learn 可选
```

15.2 前端

MVP：

```
React
TypeScript
D3.js
Vite
Tailwind
Zustand 或 Redux
```


大仓库优化：

Canvas / WebGL
虚拟滚动
按时间范围懒加载
按 zoom level 聚合

15.3 Agent 层

LLM for explanation
LLM for cluster labeling
LLM reviewer for semantic validation
Rule-based validator for deterministic checks

16. 项目里程碑

Milestone 1: 本地 Git 元数据抽取

交付：

commits.jsonl
refs.json
tags.json
file_changes.jsonl

验收：

能在任意本地 repo 上运行
commit 数量与 git log 一致
CLOC 统计可复现

Milestone 2: DAG + Repository History IR

交付：

dag.json
commits.jsonl
basic_ir_schema.md

验收：

parent-child 关系正确
merge commit 可识别
branch/tag/ref 可解析

Milestone 3: Stem + Stem Segment 编译

交付:

stems.json
stem_segments.json

验收:

main stem 正确
non-main stems 可解释
stem lifetime 正确
支持 stem 被 merge 后继续发展的情况

Milestone 4: Merge Episode + CSM

交付:

merge_episodes.json
csm_capsules.json

验收:

每个 merge episode 可用 git 命令复现
CSM-source 不重复、不遗漏
CSM 只吸收已 merge segment

Milestone 5: 基础验证与非图形报告

交付:

validation_report.md
repo_overview.md
stems.md

merge_episodes.md
csm_capsules.md

验收:

不看图也能理解仓库主要历史结构
每个结论都有 evidence
能发现 stem/CSM 的基本不一致

Milestone 6: 可视化 MVP

交付:

Simplified Stem Timeline
Overview Cards
Selected Object Summary
Evidence Table

验收:

可以浏览 main/non-main stems
可以点击 stem 查看证据
可以点击 CSM-base 查看 source commits
可以按时间和文件过滤

注意: 这一阶段不要求完整 commit clustering 和 comparison view。

Milestone 7: Context Chat MVP

交付:

选中 stem 后可追问
选中 CSM-base 后可追问
选中 evidence object 后可追问
LLM 回答包含 evidence
LLM 可返回 highlight / focus / open_evidence actions

验收:

用户点击图形对象后, 对话框能自动理解“它”指代的对象
用户可以连续追问至少 3 轮, 不丢失 selected object 上下文

每个回答都能跳转到 supporting commits/files
对话能反向操作图形，例如高亮、过滤、打开证据表

Milestone 8: Commit Clustering + Hotspot

交付：

```
clusters.json  
hotspots.json  
cluster_report.md  
hotspot_report.md
```

验收：

```
cluster 有作者、文件、关键词、时间范围、CLOC  
outlier commit 可被检测  
cluster label 有证据链  
hotspot 可追溯到相关 commits/clusters
```

Milestone 9: 增强可视化与 Cluster Context Chat

交付：

```
Cluster Timeline  
Cluster Detail View  
Hotspot View  
Context Chat 支持 cluster / hotspot
```

验收：

```
点击 cluster 后可解释主题  
点击 hotspot 后可解释原因  
能展开 supporting commits/files  
能通过对话过滤相关历史
```

Milestone 10: Comparison View

交付：

Comparison View
open_comparison action
release/stem/cluster comparison

验收：

可以比较两个 release
可以比较两个 stem
可以比较两个 cluster
比较结果包含作者、文件、关键词、CLOC、commit type
Context Chat 可解释比较结果

Milestone 11: GitHub / PR / Issue 集成

交付：

pull_requests.json
issues.json
reviews.json
release_notes.json

验收：

CSM 可关联 PR 信息
cluster label 可利用 PR title/body
release slice 可利用 release notes

Milestone 12: 增量更新与大仓库优化

交付：

incremental_update
DuckDB/SQLite index
frontend lazy loading
large repo performance report

验收：

新增 commits 可增量处理
大仓库无需全量重算
前端在大仓库上仍可交互

17. MVP 定义

第一个可用版本需要做到：

输入：本地 Git repo

输出：

- repo_overview.md
- stems.json
- stem_segments.json
- merge_episodes.json
- csm_capsules.json
- validation_report.md
- simplified stem timeline UI
- context chat for selected stem / CSM

MVP 成功标准：

1. 能看清 main stem 和主要 non-main stems。
2. 能识别每个 stem 的生命周期。
3. 能识别 stem 被 merge 后继续发展的情况。
4. 能识别 merge episode。
5. 能把 CSM-base 和 source commits 对应起来。
6. 每个抽象都有 evidence。
7. Validator 能检查抽象是否自治。
8. 用户可以通过 UI 获得总体历史形态。
9. 用户可以通过 Context Chat 持续追问选中对象。
10. Context Chat 能基于 evidence 回答，而不是基于截图猜测。

18. 风险与应对

风险 1：branch 不等于 feature

应对：

stem type 只作为 candidate
所有 semantic label 都附 evidence 和 confidence

风险 2: squash merge 破坏拓扑

应对:

本地 Git 无法恢复的信息标记为 unknown
接入 PR API 后再补充上下文

风险 3: commit message 噪声大

应对:

label 不只依赖 message
结合 file path、directory、author、time、PR title

风险 4: 大仓库性能差

应对:

先离线编译
后续引入增量更新
使用 SQLite/DuckDB
前端使用虚拟化和懒加载

风险 5: agent 过度解释

应对:

agent 只能基于 IR 生成解释
每个解释必须带 supporting commits/files
低证据解释标记 low confidence

风险 6: 用户面对图形感到茫然

应对:

默认展示 Overview Cards, 而不是完整复杂图
图形按 zoom level 渐进展开

Context Chat 提供 suggested questions
用户通过点击对象和持续追问进入图形

风险 7: Context Chat 丢失指代上下文

应对:

维护 selected_object
维护 pinned_objects
维护 conversation_state
每轮问题都显式传入 view_state 和 selected object

19. 推荐开发顺序

推荐顺序如下:

1. Git metadata extractor
2. DAG builder
3. Repository History IR schema
4. stem compiler
5. stem segment compiler
6. merge episode compiler
7. CSM capsule compiler
8. deterministic validation rules
9. non-visual Markdown report
10. simplified visualization MVP
11. Context Chat MVP
12. commit clustering
13. hotspot analysis
14. enhanced visualization for clusters/hotspots
15. comparison view
16. GitHub/PR/issue integration
17. incremental update and large repo optimization
18. Codex skill packaging

这个顺序避免了前后矛盾:

- 不先要求用户读复杂图。
- 不先依赖 LLM 解释未验证的数据。
- 不在 cluster compiler 完成前强行做 cluster-heavy UI。
- Context Chat 先支持 stem/CSM, 再扩展到 cluster/hotspot。
- 可视化始终基于 IR, 而不是截图或模型猜测。

20. 长期目标

长期目标不是复制某个 Git 可视化工具，而是构建一个适合 agent 时代的代码库历史理解系统：

```
Git history compiler
+ structured IR
+ validator agent
+ semantic explanation agent
+ evidence-first reports
+ guided visualization
+ context-aware chat
+ queryable repository memory
```

最终系统应该能够帮助用户回答：

```
这个仓库如何演化？
当前代码背后的主要历史上下文是什么？
哪些模块长期高频变化？
某个文件为什么存在、为什么频繁变化？
某次 merge 实际合入了什么？
某个 release 包含哪些主要开发主题？
哪些历史信息足以支持当前代码理解？
```

系统的最终形态应是：

```
一个可以被看见、被追问、被验证的 Git repository memory system。
```